

Implementation and Verification of a 32-Tap FIR Low-Pass Audio Filter Using Verilog HDL

Abstract

This paper presents the implementation of a 32-tap FIR low-pass filter for audio signal processing using Verilog HDL. The filter is simulated and verified in ModelSim, while MATLAB is used to evaluate its filtering performance. The simulation results indicate that the implemented filter effectively suppresses high-frequency noise and preserves the desired low-frequency components. The agreement between MATLAB and ModelSim results verifies the correctness of the proposed implementation.

1. Introduction

The rapid advancement of Very Large Scale Integration (VLSI) technology has accelerated the adoption of digital signal processing (DSP) in numerous applications [1], [2], [3]. To meet the increasing demands for high performance and reliability, hardware description languages (HDLs) and software-based design tools have become essential for implementing and verifying digital systems.

In practical applications, audio signals are often affected by high-frequency noise during recording, transmission, and processing stages. The presence of these unwanted components can significantly degrade audio quality, resulting in signal distortion and reducing the original clarity of the sound. Therefore, developing an effective signal processing solution to eliminate undesired frequency components while preserving the useful audio information has become a critical requirement.

FIR filters can be categorized into several types depending on the application [1], [2], including Low-Pass Filters (LPF), High-Pass Filters (HPF), Band-Pass Filters (BPF), and Band-Stop (Notch) Filters. In addition, other filters such as tone control filters, peak filters, crossover filters, and all-pass filters are also available. Each type is designed to perform a specific frequency-selective operation depending on the application. Among these filters, the FIR Low-Pass Filter is selected in this work because it effectively suppresses high-frequency noise while preserving low-frequency audio components.

- **Low-Pass Filter (LPF):** Allows low-frequency components to pass while attenuating high-frequency signals. It is commonly used for extracting bass signals for subwoofers or reducing high-frequency noise.

The main objective of this project is to implement, simulate, and verify a 32-tap FIR low-pass digital filter for audio signal processing. A 32-tap FIR filter provides a good balance between filtering performance and implementation complexity. It offers sufficient attenuation of high-frequency noise while maintaining a relatively simple hardware architecture suitable for HDL implementation and simulation. The project mainly focuses on the implementation and functional verification of the filter using MATLAB/Simulink and Verilog HDL. Specifically, the project focuses on the following tasks:

- **Verilog HDL Implementation:** Implement a 32-tap FIR filter in Verilog HDL using a predefined set of filter coefficients.
- **MATLAB/Simulink Verification:** Construct a MATLAB/Simulink model using the same filter coefficients to analyze the filter response in both the time domain and the frequency domain.
- **Functional Verification:** Perform functional simulation of the Verilog implementation in ModelSim
- **Performance Evaluation:** Evaluate the filtering performance by analyzing the attenuation of high-frequency components while preserving the desired low-frequency signal.

2. Theoretical Background

2.1 Difference Equation

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n - k] \quad [1]$$

Where:

- $y[n]$: Output signal
- $x[n]$: Input signal
- $h[k]$: Filter coefficients (impulse response)
- N : Number of filter taps (filter order + 1)
- k : Coefficient index

Purpose

This formula is used to calculate the output signal of a Finite Impulse Response (FIR) filter [1]. The output is obtained by multiplying the current and previous input samples with the filter coefficients and then summing the results. FIR filters are commonly used to reduce noise and improve signal quality while maintaining system stability.

2.2 Mathematical Model of the 32-Tap FIR Low-Pass Filter

2.2.1 With 32 filter coefficients, the equation can be expanded as follows:

$$y[n] = h_0 \cdot x[n] + h_1 \cdot x[n-1] + h_2 \cdot x[n-2] + \dots + h_{30} \cdot x[n-30] + h_{31} \cdot x[n-31]$$

Where $h = \{h_0, h_1, h_2, \dots, h_{31}\}$ is the predefined coefficient set used in this work. During each sampling cycle, a new input signal is shifted into the shift register, and the older samples are shifted one position to the right. Each sample is then multiplied by its corresponding coefficient and accumulated to generate the output signal.

2.2.2 Z-Domain Transfer Function:

In the Z-domain, the transfer function of the FIR filter is expressed as:

$$H(z) = \sum h[k] \cdot z^{-k} \text{ (with } k \text{ from } 0 \text{ to } 31) \quad [1]$$

This transfer function characterizes the frequency response of the filter and serves as the foundation for analyzing the system's behavior.

2.2.3 Coefficient Symmetry:

The implemented FIR filter uses a symmetric coefficient set, resulting in a linear-phase response [1]. Consequently, its coefficients exhibit strict symmetry:

$$h[k] = h[31 - k]$$

where $k = 0, 1, \dots, 15$. This symmetric property ensures that the filtered signal's waveform remains undistorted. Furthermore, it allows for a reduction in the number of required hardware multipliers if a folded architecture is implemented.

2.2.4 Frequency Response:

The frequency response of the FIR filter is obtained by evaluating the transfer function on the unit circle, substituting $z = e^{j\omega}$:

$$H(e^{j\omega}) = \sum h[k] \cdot e^{-j\omega k} \text{ (with } k \text{ from } 0 \text{ to } 31) \quad [1]$$

Where:

- $H(e^{j\omega})$: Frequency response of the filter.
- ω : Normalized angular frequency (rad/sample).
- $h[k]$: FIR filter coefficients.

The filter exhibits a low-pass frequency response, allowing frequencies below the cutoff frequency to pass while attenuating higher-frequency components[3].

2.2.5 Group Delay :

A direct consequence of the exact linear phase characteristic is a constant group delay across all frequencies. The group delay (τ) for an N-tap FIR filter is determined by:

$$\tau = \frac{N-1}{2}$$

[1]

For the 32-tap FIR filter used in this project, the system introduces a constant group delay of $\tau = 15.5$ samples. This constant delay is crucial in audio processing as it ensures that all frequency components arrive at the output simultaneously, preventing temporal smearing.

2.3 Direct Form Block Diagram Description

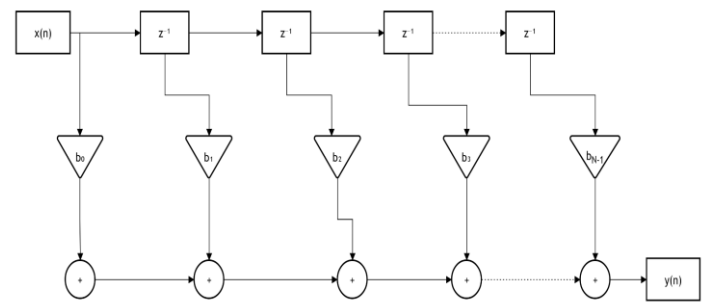


Figure 2.3 : Direct Form Block Diagram

Figure 2.3 illustrates the Direct Form hardware architecture of the N-tap FIR filter (for this project, $N = 32$). This diagram is a visual representation of the filter's convolution mathematical equation, clearly demonstrating the data flow through the system. The structure consists of the following basic hardware components:

- **Input signal $x(n)$:** Represents the digital audio data sample at the current time index n .
- **Delay Elements (z^{-1}):** Act as Shift Registers in the Verilog code. Each block holds the data for exactly one clock cycle to generate past signal samples such as $x(n-1)$ and $x(n-2)$, respectively.
- **Multipliers (Triangle symbols):** Perform the multiplication between the signal samples and the filter coefficients $b_0, b_1, \dots, b_{n-1}$. These b which predefined FIR filter coefficients.
- **Adders (Circle symbols with +):** Function as Accumulators. These blocks sum all the results from the multiplier branches to produce a single final value.
- **Output signal $y(n)$:** Represents the audio data sample after the multiply-accumulate process, where high-frequency noise components have been significantly attenuated.

Through this diagram, the Multiply-Accumulate (MAC) process is coherently demonstrated. Utilizing the Direct Form architecture ensures that the hardware source code closely adheres to the theoretical foundation of Digital Signal Processing (DSP), while also making the management of the data flow within ModelSim more intuitive and easier to control.

3. Experimental Results and Verification with Matlab and ModelSim

3.1 MATLAB/Simulink

3.1.1 Simulation Objective

To verify the functionality of the implemented 32-tap FIR filter, MATLAB/Simulink was employed as a reference verification platform [3],[4]. The filter coefficients used in the Simulink model were taken directly from the Verilog implementation (fir32_lowpass.v) to ensure consistency between the algorithmic model and the hardware description. The simulation focused on evaluating the filter response in both the time domain and the frequency domain before comparing the results with the Verilog HDL simulation in ModelSim.

3.1.2 Simulink Model

The simulation model consists of two discrete sine wave generators with frequencies of 1 kHz and 10 kHz, representing the desired low-frequency signal and high-frequency noise, respectively. These two signals are summed and applied to a Discrete FIR Filter block. The filter output is then observed using a Scope and a Spectrum Analyzer. Figure 3.1.1 illustrates the complete Simulink model used in this work.

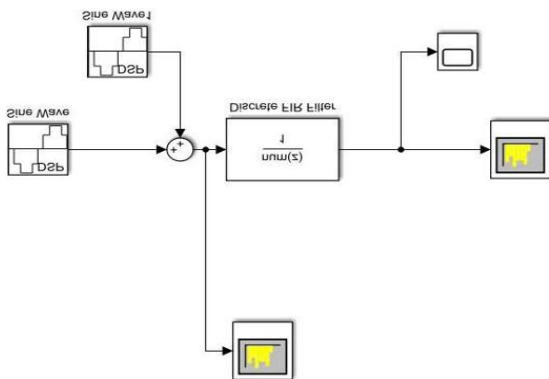


Figure 3.1.1: Simulink model of the 32-tap FIR filter.

3.1.3 Simulation Parameters

The parameters used during the simulation are listed in the table.

Parameter	Value
Sampling frequency	48 kHz
Simulation time	0.05 s
Input signal 1	1 kHz sine wave
Input signal 2	10 kHz sine wave
Signal amplitude	1
FIR filter length	32 taps
Filter coefficients	Imported from fir32_lowpass.v

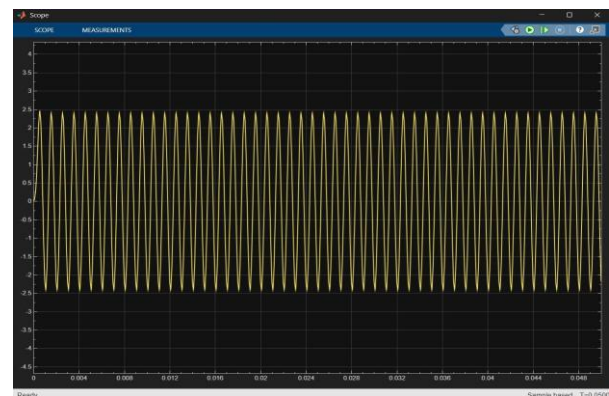


Figure 3.1.3: Output waveform of the FIR filter in the time domain.

Time-domain output waveform of the 32-tap FIR filter. After the composite input signal containing 1 kHz and 10 kHz frequency components passes through the filter, the output reaches a steady state after an initial transient response. The waveform illustrates the filtering performance of the implemented FIR filter using the predefined coefficient set.

3.1.4 Time-Domain Analysis

Figure 3.1.4 presents the input and output signals in the time domain.

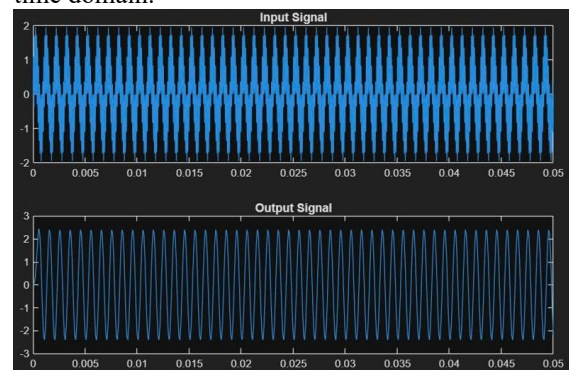


Figure 3.1.4: Input and output signals of the FIR filter in the time domain.

The input signal is composed of two sinusoidal components at 1 kHz and 10 kHz. After passing through the FIR filter, the output waveform becomes smoother while maintaining the overall signal shape. A short transient response appears at the beginning of the waveform due to the initialization of the FIR delay line, after which the filter reaches a steady-state response.

This figure shows:

- Time Domain waveform.
- The FIR filter is operating correctly.
- Output signal is different from input signal.

3.1.5 Frequency-Domain Analysis

The frequency spectra before and after filtering were analyzed using the Spectrum Analyzer block.

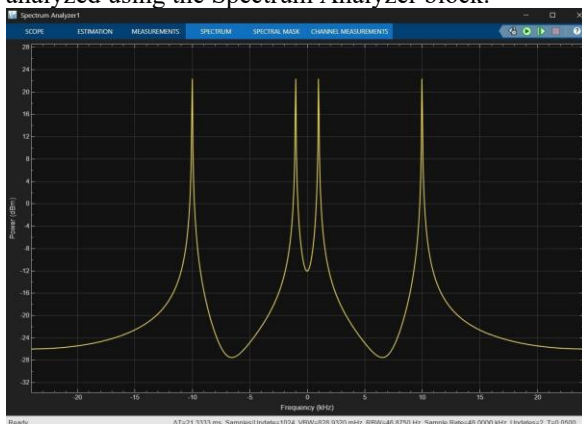


Figure 3.1.5a Frequency spectrum of the input signal.

The input spectrum clearly shows two dominant frequency components located at approximately 1 kHz and 10 kHz, corresponding to the generated sinusoidal signals.

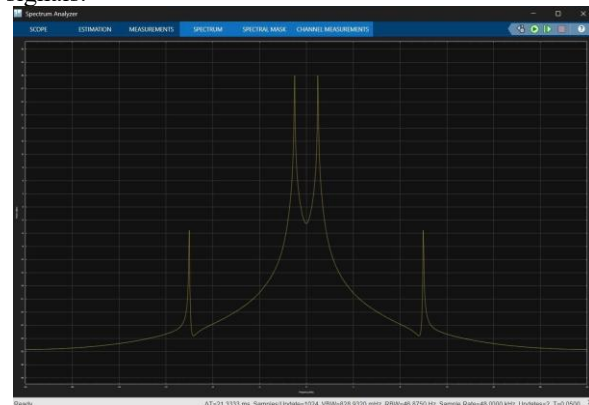


Figure 3.1.5b Frequency spectrum of the filtered output signal.

After filtering, the component around 10 kHz is noticeably attenuated, while the 1 kHz component remains dominant. This result indicates that the implemented FIR filter suppresses higher-frequency components according to the coefficient set implemented in the Verilog design.

3.2 Modelsim

3.2.1 System Operation Flowchart

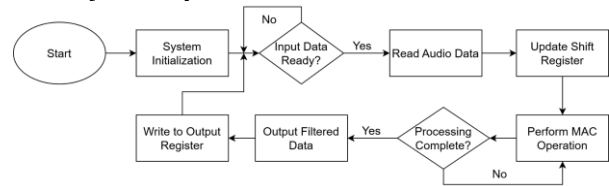


Figure 3.2.1: System operation flowchart of the FIR filter in the simulation environment.

To provide a clear understanding of the digital logic before delving into the Verilog code, the system's operational flow is illustrated in Figure 3.2.1. Unlike a sequential software program, this hardware-described system operates in a continuous loop driven by the testbench clock signal during simulation.

Upon startup, the system initialization block resets all internal registers, variables, and the delay line. The system then enters a continuous monitoring state, checking for the availability of new input data provided by the testbench stimulus. Once a new digital audio sample is clocked in, the data is shifted into the delay line (Shift Register) to update the sample history. Subsequently, the core FIR filter performs the Multiply-Accumulate (MAC) computation. By utilizing structural coefficient symmetry, the system calculates the outputs efficiently within minimal clock cycles. Finally, the filtered data is normalized to prevent overflow and outputted for waveform observation in ModelSim. The process immediately loops back to await the next incoming sample from the testbench, ensuring continuous digital signal processing throughout the simulation runtime.

3.2.2 Core Verilog Modules

In this work, a 32-tap FIR filter was adopted as a representative configuration to demonstrate the hardware implementation of an FIR filter. This filter order provides a reasonable trade-off between filtering performance and implementation complexity while keeping the Verilog design and simulation manageable [3].

To demonstrate the operation of the Low-pass Filter, it is necessary to focus on the key code blocks of the Verilog program, as each module performs a specific function in signal processing.

★ Filter Coefficient Declaration Block

```

coeff[0]=1;
coeff[1]=1;
coeff[2]=2;
coeff[3]=2;
coeff[4]=3;
...
coeff[15]=14;
coeff[16]=14;
coeff[17]=13;
...
  
```

coeff[31]=1;

This block defines the coefficients of the 32-tap FIR filter. By observing the coefficient values, it can be seen that they are arranged symmetrically around the center position. The coefficient values are predefined and remain constant throughout the implementation.

$$h(n) = h(N - 1 - n)$$

Examples:

- coeff[0] = coeff[31] = 1
- coeff[1] = coeff[30] = 1
- coeff[2] = coeff[29] = 2
- coeff[15] = coeff[16] = 14

This symmetry characteristic produces a Linear Phase response [1],[3], meaning that different frequency components of the signal experience the same amount of time delay. This property is particularly important in audio processing because it helps prevent signal distortion after filtering.

In addition, the coefficients gradually increase from both ends toward the center and then decrease symmetrically. The corresponding low-pass characteristic is confirmed through MATLAB/Simulink and ModelSim verification. The coefficients in the central region have larger weights, allowing low-frequency components to be preserved more effectively, while high-frequency components are attenuated.

★ Sample Memory Block

```
for(i=N-1;i>0;i=i-1)
delay[i]<=delay[i-1];
```

```
delay[0]<=xin;
```

This block functions as a Shift Register. Its purpose is to store previous input signal samples for use in the convolution process of the FIR filter. Assuming that the input data sequence is x(0), x(1), x(2), and x(3), the operation during each clock cycle is as follows:

Clock	delay [0]	delay [1]	delay [2]	delay [3]
t0	x(0)	0	0	0
t1	x(1)	x(0)	0	0
t2	x(2)	x(1)	x(0)	0
t3	x(3)	x(2)	x(1)	x(0)

After a certain period of operation, the system will store the sequence: x(n), x(n-1), x(n-2), ..., x(n-31). Storing multiple consecutive samples is necessary because an FIR filter does not process only the current sample; instead, it performs calculations based on both the current and previous input samples.

★ Multiply and Accumulate (MAC) Block

```
sum=0;
```

```
for(i=0;i<N;i=i+1)
sum=sum+delay[i]*coeff[i];
```

This is the most important block of the FIR filter because it performs the convolution operation between the input signal and the filter coefficients. The mathematical equation is as follows:

$$y(n) = \sum_{i=0}^{N-1} h(i) \cdot x(n - i)$$

This computation is performed for every input sample at each clock cycle. If the signal changes slowly (low-frequency signal), for example: 5, 6, 5, 6, 5, 6 → the values are relatively similar; therefore, after being multiplied by the coefficients and summed together, the result still maintains a relatively large magnitude.

If the signal changes rapidly (high-frequency noise), for example: 5, -5, 5, -5, 5, -5 → after multiplication and accumulation, the positive and negative values tend to cancel each other out, resulting in a reduction in amplitude.

This is the fundamental reason why a low-pass filter can suppress high-frequency noise while preserving the desired signal components.

★ Output Data Normalization Block

```
yout<=sum>>>6;
```

After performing the multiplication and accumulation process for all 32 elements, the resulting sum value can become significantly large. For example, assuming x = 100 and the sum of the coefficients is approximately 156, then the calculated result becomes sum = 15600. This value exceeds the range of smaller data representations and therefore requires normalization using the command: ">>> 6". In this case, the sum value performs an arithmetic right shift by six bits (>>>6), equivalent to division by 2⁶ while preserving the sign in order to reduce the output amplitude, prevent data overflow, and bring the signal back into an appropriate operating range.

3.3 ModelSim Verification Results

The ModelSim simulation waveform contains the following signals:

- *clk*: the system clock signal.
- *rst_n*: the active-low reset signal. Initially, the reset is set to logic level 0 and then switched to logic level 1, allowing the system to begin normal operation.
- *x_{in}*: the input signal containing high-frequency noise.
- *x_{clean}*: the desired original signal (without noise).
- *y_{out}*: the output signal after passing through the FIR filter.

- n : a counter variable used during the simulation process.

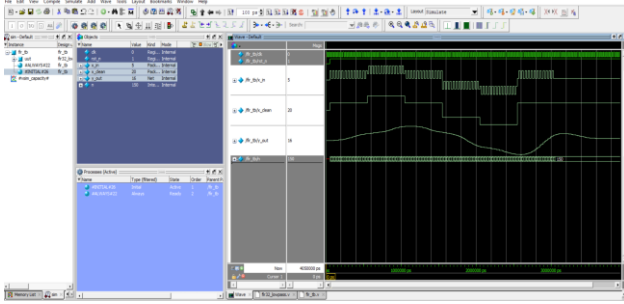


Figure 3.3. ModelSim simulation waveform.

Observation of the waveform shows that the x_{in} signal exhibits numerous rapid and continuous fluctuations. These fluctuations represent the high-frequency noise components added to the original signal. Due to the presence of noise, the input waveform becomes unstable and contains many small pulses. In contrast, the x_{clean} signal changes more gradually and appears relatively smooth, representing the desired original signal. After the x_{in} signal passes through the filter, the y_{out} signal demonstrates a smoother waveform, with the rapid fluctuations being significantly reduced. The signal amplitude follows the general trend of x_{clean} , and the output signal closely resembles the original desired signal.

It can be observed that during the time interval from 100000 ps to 300000 ps, the x_{in} signal still contains high-frequency fluctuations; however, the y_{out} signal changes gradually according to the overall trend of the signal. This indicates that the high-frequency components have been significantly attenuated. The reason is that the FIR filter performs the convolution operation:

$$y(n) = \sum_{i=0}^{31} h(i) \cdot x(n - i)$$

Low-frequency components change slowly; therefore, when accumulated, they tend to reinforce each other. In contrast, high-frequency noise components change rapidly, causing positive and negative values to cancel each other out. In addition, the y_{out} signal exhibits a slight delay compared to x_{clean} . This is a normal characteristic of an FIR filter because multiple previous samples must be stored in the shift register before the computation can be performed. This delay is referred to as the filter's group delay.

From the simulation results, it can be concluded that the 32-tap FIR Low-pass filter operates correctly as intended, preserving the low-frequency signal components while significantly suppressing high-frequency noise, thereby allowing the output signal to closely resemble the original input signal.

4. Discussion

4.1 Comparison

Characteristic	Before Filtering (x_{in})	After Filtering (y_{out})
Signal Smoothness	Contains many small fluctuations	Smoother
Noise Components	Contains significant high-frequency noise	Noise is significantly reduced
Stability	Changes rapidly and is unstable	Changes gradually and becomes more stable
Similarity to Original Signal	Deviates due to noise	Closely resembles x_{clean}
Delay	No delay	A slight delay is present

4.2 Advantages and Limitations

Strengths:

- **Linear Phase Response:** Due to the symmetry of the 32 coefficients ($h(n) = h(N-1-n)$), the FIR filter ensures that all frequency components of the audio signal are delayed by the exact same amount of time. This is extremely crucial in audio processing as it prevents phase distortion in the output signal, thereby preserving the fidelity of the original audio.
- **Absolute Stability:** Unlike IIR (Infinite Impulse Response) filters, the FIR structure does not contain feedback loops. Consequently, the system completely eliminates the risk of mathematical divergence or instability during operation.
- **Significant Noise Reduction Effectiveness:** Results from the ModelSim simulation environment prove that the system accurately executes the convolution operation (MAC), thoroughly attenuating high-frequency noise fluctuations while maintaining the integrity of the desired audio signal envelope.

Limitations:

- **High Computational Resource Consumption:** With the 32-tap Direct Form structure, the algorithm requires 32 multiplication and addition operations for each signal sample. If deployed on an actual FPGA, the system would consume a substantial amount of hardware resources (DSP slices and Logic elements).
- **Group Delay:** To process 32 consecutive data samples, the system requires time to fill the shift registers. This process inevitably creates a latency between the input and output. Although completely acceptable in simulation, it may need to be considered in systems requiring ultra-low latency.
- **Hardcoded Coefficients:** Currently, the coefficients are statically configured within the Verilog code. Modifying the cutoff frequency requires the user to regenerate the

coefficient set using an appropriate filter design tool before recompiling the hardware.

4.3 Future Directions

Although the system operated accurately in the simulation environment, this research opens up several potential development directions for practical application:

- **Hardware Implementation:** The next critical step is to synthesize the source code and program it onto a physical FPGA board. The system can be integrated with the I2S protocol to interface directly with an Audio Codec chip, enabling real-time audio noise filtering and processing.
- **Architecture Optimization:** To address the limitation of high resource consumption, the system architecture can be improved. By leveraging the symmetry of the filter coefficients, a Folded Architecture can be applied to reduce the required number of multipliers by half while maintaining the same filter quality.
- **Programmable Coefficients:** The system could be upgraded by adding standard communication interfaces (such as SPI or I2C). This would allow an external microcontroller to load and modify the filter coefficients on-the-fly while the system is running, transforming it into a versatile filter capable of switching flexibly between Low-Pass, High-Pass, or Band-Pass depending on the audio environment requirements.

5. Conclusion

This study successfully demonstrated the implementation and verification of a 32-tap symmetric low-pass FIR filter using Verilog and MATLAB. MATLAB/Simulink was utilized for functional verification of the predefined coefficient set, while ModelSim was used to verify the functionality and performance of the system through simulation. The simulation results confirmed that the FIR filter effectively preserved low-frequency signal components while significantly reducing high-frequency noise. Furthermore, the output signal showed a close resemblance to the original desired signal, with only a small delay introduced due to the filter structure. The findings indicate that the implemented filter performs according to the expected theoretical behavior and provides a solid foundation for future development and implementation of more advanced digital signal processing applications.

References

- [1] A. V. Oppenheim and R. W. Schaffer, Discrete-Time Signal Processing, 3rd ed. Pearson, 2010.
- [2] M. A. A. Al-Dulaimi, H. A. Wahhab, and A. A. Amer, "Design and Implementation of Communication Digital FIR Filter for Audio Signals on the FPGA Platform," Journal of Communications, vol. 18, no. 2, pp. 89–96, Feb. 2023, doi:10.12720/jcm.18.2.89-96.
- [3] R. A. Losada, Practical FIR Filter Design in MATLAB, Rev. 1.0, The MathWorks, Inc., Natick, MA, USA, Mar. 31, 2003.

- [4] MathWorks, DSP System Toolbox Documentation
- [5] S. Bhavani, A. Ganga Bhargavi, B. Chinna Reddaiah, D. Divya, and A. Asha, "Design of FIR Filter Based on HDL Coder," International Journal of Engineering Technology and Management Sciences, vol. 7, Special Issue 1, pp. 84–86, Apr. 2023.